

UNIVERSITY OF MYSORE



DISSERTATION ON

“Real Time System For Sign Language Recognition”

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR
THE AWARD OF THE

**BACHELOR OF COMPUTER APPLICATION
VIDHYAASHRAM FIRST GRADE COLLEGE**

JAYALAKSHMIPURAM,MYSORE-570012.



**UNDER THE GUIDANCE OF
KOUSHIK K S**

DEPARTMENT OF COMPUTER SCIENCE

SUBMITTED BY:

1. SANDEEP S V

U01CJ23S0037

2.ROSHAN LOUIS MENEZES

U01CJ23S0010

3. MUHAMMED ADNAN

U01CJ23S0003

4.JALAM SINGH CHOUHAN

U01CJ23S0024

“Real Time System For Sign Language Recognition”

DISSERTATION REPORT

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENT
FOR THE AWARD OF THE
BACHELOR OF COMPUTER APPLICATION
VIDHYAASHRAM FIRST GRADE COLLEGE

JAYALAKSHMIPURAM,MYSORE-570012.

UNIVERSITY OF MYSORE



UNDER THE GUIDANCE OF
KOUSHIK K S
DEPARTMENT OF COMPUTER SCIENCE

SUBMITTED BY:

1. SANDEEP S V

U01CJ23S0037

2.ROSHAN LOUIS MENEZES

U01CJ23S0010

3. MUHAMMED ADNAN

U01CJ23S0003

4. JALAM SINGH CHOUHAN

U01CJ23S0024

**DEPARTMENT OF COMPUTER SCIENCE
VIDHYAASHRAM FIRST GRADE COLLEGE
UNIVERSITY OF MYSORE**



CERTIFICATE

This is to certify that **Sandeep SV, Roshan Louis Menezes, Muhammed Adnan** and **Jalam Singh Chouhan** have successfully completed their final semester dissertation work entitled “**Real Time System For Sign Language Recognition**” in partial fulfilment of the requirements for the award of degree of ‘Bachelor of Computer Application’ of University of Mysore, during the academic year 2026. This is an authentic work, free from plagiarism and not submitted to any degree or so.

Internal Guide

KOUSHIK K S

Examiners:

- 1.
- 2.

Date of Examination:

ACKNOWLEDGEMENT

I will remain indebted to our Principal **Kushy A M** for giving us an opportunity to carry out our dissertation work.

We thank **Vidhyaashram First Grade College** for fostering us this opportunity to elevate our academics, provided with the best ambience and the infrastructure.

I take the opportunity to express our deep sense of gratitude to our dissertation guide **Koushik K S, Professor, Department of Computer Science**, for his inspiring guidance and valuable suggestions throughout the dissertation work.

We thank all the teaching and non-teaching staff for their coherent support.

Sincere thanks to all friends and family who have patiently extended all sorts of help for accomplishing this undertaking.

1. SANDEEP S V

U01CJ23S0037

2. ROSHAN LOUIS MENEZES

U01CJ23S0010

3. MUHAMMED ADNAN

U01CJ23S0003

4. JALAM SINGH CHOUHAN

U01CJ23S0024

DECLARATION

We here by declare that the dissertation work entitled “ **Real Time System For Sign Language Recognition**” submitted by us to **VIDHYAASHRAM FIRST GRADE COLLEGE, MYSORE** “in partial fulfillment of the curriculum prescribed for the award of the degree, Bachelor of Computer Application (BCA) is a record of a dissertation work carried out by us under the guidance of **Koushik K S.**

We further declare that the work reported in this dissertation has been submitted and will not be submitted by her/him in a part or in full ,for the award of any other degree or diploma in this institute or any institute of university.

Date:

Place:

1. SANDEEP S V

U01CJ23S0037

2. ROSHAN LOUIS MENEZES

U01CJ23S0010

3. MUHAMMED ADNAN

U01CJ23S0003

4. JALAM SINGH CHOUHAN

U01CJ23S0024

ABSTRACT

Deaf and speech-impaired individuals face significant communication challenges in their daily lives, as they primarily rely on sign language, which is not widely understood by the majority of the population that uses spoken and written language for communication. This communication gap creates a strong need for technological tools capable of accurately interpreting sign language. Although considerable research has been conducted on global sign languages, there is still scope for improving intelligent and efficient recognition systems. This project focuses on developing a machine learning-based system for recognizing American Sign Language (ASL) gestures. A dataset was created using a combination of two-dimensional and three-dimensional images representing selected static ASL alphabet gestures. The MediaPipe framework was implemented to detect and extract hand landmarks from the gesture images, capturing key positional features of the fingers and palm. These extracted features were used to train a feed-forward neural network model for gesture classification. The trained model achieved high accuracy in recognizing the selected ASL gestures. The results demonstrate that the proposed approach is effective for sign language recognition and can be further extended to include dynamic gestures, full alphabet sets, and real-time sign-to-text or sign-to-speech conversion systems

CONTENTS

CHAPTER 1: INTRODUCTION.....	9
1.1 INTRODUCTION TO THE SYSTEM.....	9
1.2 AIM.....	10
1.3 OBJECTIVES.....	11
1.4 SCOPE OF PROJECT.....	12
1.5 APPLICATIONS.....	13
1.6 ADVANTAGES.....	14
 CHAPTER 2: LITERATURE SURVEY.....	 15
2.1 RELATED WORK.....	15
 CHAPTER 3: SYSTEM ANALYSIS.....	 18
3.1 HARDWARE REQUIREMENTS.....	18
3.2 SOFTWARE REQUIREMENTS.....	19
 CHAPTER 4: SYSTEM DESIGN AND UI.....	 20
4.1 SYSTEM ARCHITECTURE.....	20
4.2 USER INTERFACE	22
4.3 USER INTERFACE DESIGN.....	23

CHAPTER 5: METHODOLOGY.....	24
5.1 DATA COLLECTION.....	24
5.2 DATA PREPROCESSING.....	25
5.3 DATA VISUALIZATION.....	26
5.4 MODEL BUILDING.....	29
CHAPTER 6: FUTURE SCOPE.....	31
6.1 POTENTIAL IMPROVEMENTS.....	31
CHAPTER 7: IMPLEMENTATION.....	33
CHAPTER 8: TEST CASES.....	39
7.1 PURPOSE OF TEST CASES.....	39
7.2 TEST CASE REPORT.....	40
CHAPTER 9: GESTURES.....	41
CHAPTER 10: CONCLUSION.....	44
CHAPTER 11: REFERENCES.....	45

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION TO THE SYSTEM

The proposed system is a real-time Sign Language Recognition System designed to interpret hand gestures and convert them into meaningful text output. The system captures live video input using a webcam and processes it using computer vision techniques to detect and analyze hand movements. The system uses MediaPipe for accurate hand landmark detection and a machine learning model such as Feedforward Neural Network (FNN) or LSTM for gesture classification.

It focuses on recognizing predefined static gestures efficiently and displaying the corresponding output in real time. The main objective of this system is to reduce the communication gap between individuals who use sign language and those who do not understand it. The system is designed to be simple, fast, and user-friendly, making it suitable for real-time applications and future enhancements.

1.2 AIM

The main aim of this project is to develop an efficient and real-time Sign Language Recognition System that can accurately detect hand gestures and convert them into readable text using computer vision and machine learning techniques. The system utilizes MediaPipe for hand landmark detection and a trained neural network model for gesture classification. It is designed to capture real-time input through a webcam, process the gesture data, and display the predicted output instantly

This project mainly focuses on reducing the communication gap between hearing and speech impaired individuals and the general public by providing a simple, reliable, and user-friendly solution.

- To design a system that captures real-time hand gestures using a webcam
- To train a machine learning model for gesture classification
- To use MediaPipe for accurate hand landmark detection and feature extraction

1.3 OBJECTIVES

The main objective of this project is to develop a real-time Sign Language Recognition System that can accurately detect and classify hand gestures and convert them into readable text.

- To study basic sign language gestures and their representation
- To collect and create a structured dataset of hand gestures
- To preprocess and normalize gesture data for better accuracy
- To implement hand detection using MediaPipe framework
- To extract hand landmark features from input images
- To train a machine learning model (FNN/LSTM) for classification
- To achieve accurate and real-time gesture prediction
- To develop a simple and interactive user interface

1.4 SCOPE OF PROJECT

The scope of this project is limited to developing a real-time Sign Language Recognition System that focuses on identifying predefined hand gestures and converting them into text output.

- Recognition of predefined static hand gestures
- Uses single-hand detection model
- Works under controlled lighting conditions
- Provides real-time gesture prediction
- Designed for educational and assistive use
- Can be extended for advanced features in future

1.5 APPLICATIONS

- Communication aid for hearing and speech impaired individuals.
- Learning tool for sign language education.
- Human-computer interaction systems.
- Gesture-based control systems.
- Smart assistant applications.
- Assistive technology in public services.

1.6 ADVANTAGES

- Real-time gesture recognition.
- Easy to use and user-friendly.
- No additional hardware required.
- Cost-effective solution.
- Fast and accurate performance.
- Scalable for future improvements.

CHAPTER 2

LITERATURE SURVEY

2.1 LITERATURE SURVEY

Sign language recognition has gained significant attention in recent years with the advancement of machine learning and computer vision technologies. Many earlier systems used image processing techniques and Convolutional Neural Networks (CNN) to classify hand gestures from static images. Some systems used sensor-based gloves to capture hand movements, which offered high precision but were expensive and not practical for everyday use. Recent studies have focused on vision-based approaches using cameras and frameworks like MediaPipe for efficient hand tracking and landmark detection. Compared to previous methods, this project uses a real-time webcam-based system with MediaPipe and a neural network model (FNN/LSTM), making it more efficient, cost-effective, and suitable for real-time applications.

Earlier systems mainly used image processing techniques and Convolutional Neural Networks (CNN) to recognize hand gestures from static images. These methods focused on extracting features such as hand shape, edges, and contours from images. While these systems achieved good accuracy, they required large datasets and high computational power. Additionally, most of these models were limited to offline processing and were not suitable for real-time applications.

2. Sensor-Based Recognition Systems

Some research introduced glove-based systems equipped with sensors to capture hand movements, finger positions, and orientation. These systems provided high accuracy and precise gesture detection. However, they were expensive, required additional hardware, and were not convenient for daily use. Due to these limitations, sensor-based systems are less preferred in practical applications.

3. Vision-Based Recognition Systems

Recent advancements focus on vision-based systems that use cameras and computer vision techniques for gesture recognition. Frameworks like MediaPipe enable efficient hand tracking and extraction of hand landmarks in real time. In this project, a vision-based approach is used along with a machine learning model (FNN/LSTM) to achieve accurate and fast gesture recognition.

4. Models Used in Previous Studies

Author / Study	Model / Technique	Dataset Used	Key Features
Das et al. (2018)	CNN (Inception V3)	ASL image dataset	Static gestures, RGB images
Sahoo (2021)	k-NN, Naïve Bayes	ISL digit dataset (0-9)	k-NN ~98.36%
Ansari & Harit (2016)	K-means Clustering	ISL 3D dataset	~90.68% accuracy
Rekha et al. (2011)	SVM	ISL dataset	~86.3% accuracy
Bhuyan et al. (2011)	Nearest Neighbor	Skin detection, edge features	~86.3%, slow
Pugeault & Bowden (2011)	Random Forest	ISL dataset	~90% accuracy
Keskin et al. (2013)	Object Recognition	Gabor filters, depth data	High accuracy
Bhhyan et al. (2011)	ISL dataset	Depth sensor (Kinect)	~90% accuracy
Pugeault & Bowden (2011)	ASL 3D dataset	ASL 3D dataset	High accuracy
Keskin et al. (2013)	Random Forest	ASL 3D dataset	~99% accuracy
Halder & Tayade (2021)	MediaPipe + SVM	Multi-sign dataset	~99% accuracy
Proposed System (2023)	MediaPipe + SVM	Assamese dataset (2094 samples)	~99% accuracy
Proposed System (2023)	MediaPipe + Neural Network	Assamese dataset (2094 samples)	~99% accuracy
Proposed System (2023)	MediaPipe + SVM	Assamese dataset (2094 samples)	~99% accuracy
Halder & Tayade (2021)	MediaPipe + Neural Network	Assamese dataset (2094 samples)	~99% accuracy
Proposed System (2023)	MediaPipe + Neural Network	Assamese dataset (2094 samples)	99% accuracy

CHAPTER 3

SYSTEM ANALYSIS

3.1 HARDWARE REQUIREMENTS

- **5 or Equivalent processor:**

A multi-core processor such as the Intel i5 ensures the system can efficiently handle parallel tasks such as data preprocessing, model execution, and backend computation.

- **4GB RAM:**

This is sufficient for running development tools, handling datasets used for prediction, and executing the application without performance issues.

- **150MB of Disk Space :**

The application itself is lightweight. This space accounts for the core application files, models, and environment dependencies.

3.2 SOFTWARE REQUIREMENTS

1. Operating System:

- Windows 11 (or any compatible OS)

2. Development Tools / IDEs:

- Jupyter Notebook – for interactive coding and data analysis
- PyCharm – for Python script development and backend coding
- Visual Studio Code (VS Code) – for HTML, CSS, and frontend integration

3. Programming Language:

- Python 3.x

CHAPTER 4

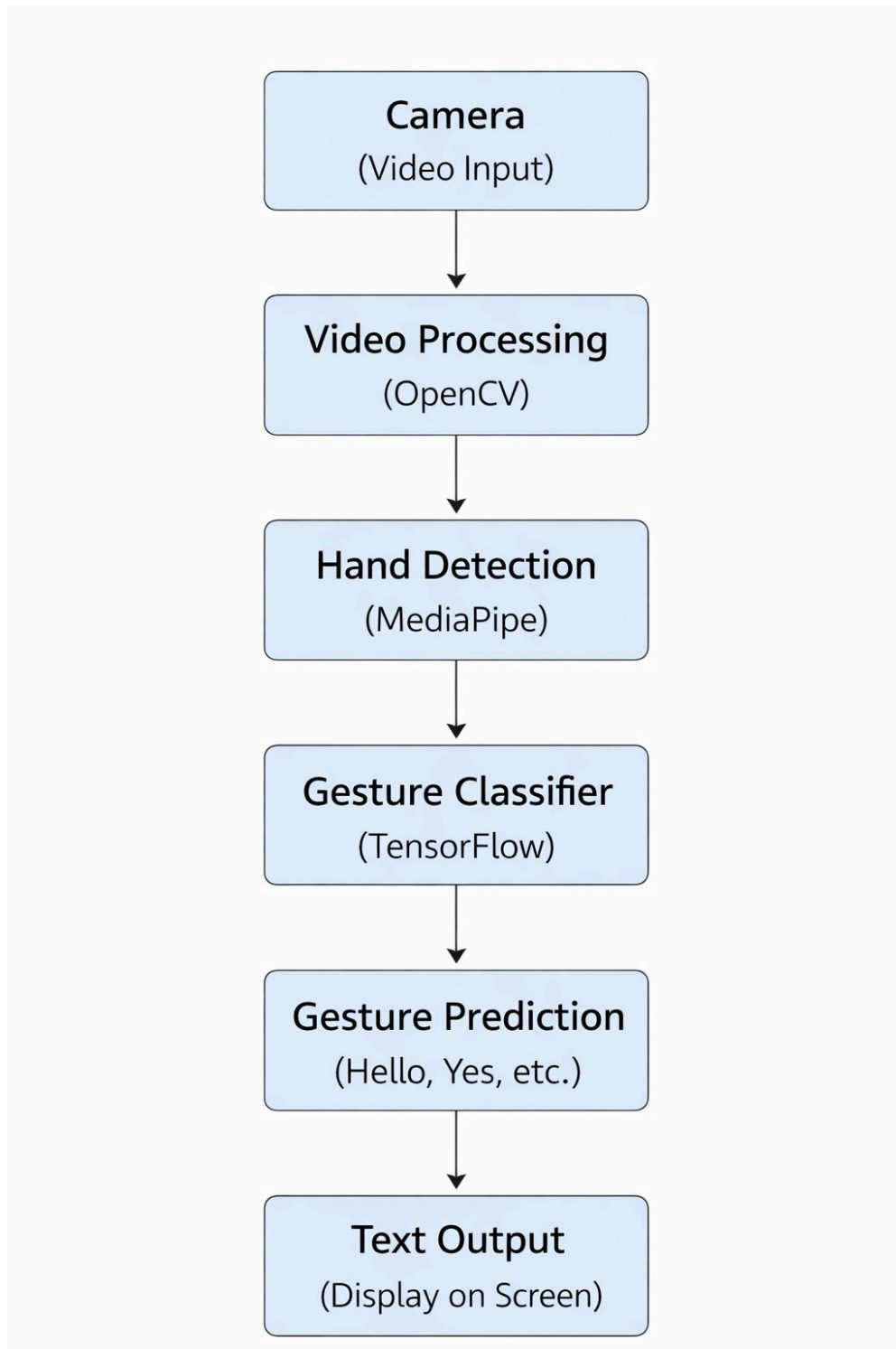
SYSTEM DESIGN

SYSTEM DESIGN

The system design focuses on developing a real-time sign language recognition system that integrates computer vision and machine learning techniques. The system begins with capturing live video input through a webcam, which is processed using OpenCV for frame extraction. MediaPipe is used to detect the hand and extract 21 landmark points representing finger positions.

4.1 SYSTEM ARCHITECTURE OVERVIEW

The system architecture consists of interconnected modules for real-time gesture recognition. Input is captured through a webcam and processed using MediaPipe for hand landmark detection. The extracted features are fed into a trained model for classification. The predicted gesture is then displayed as text output, ensuring fast and accurate performance.



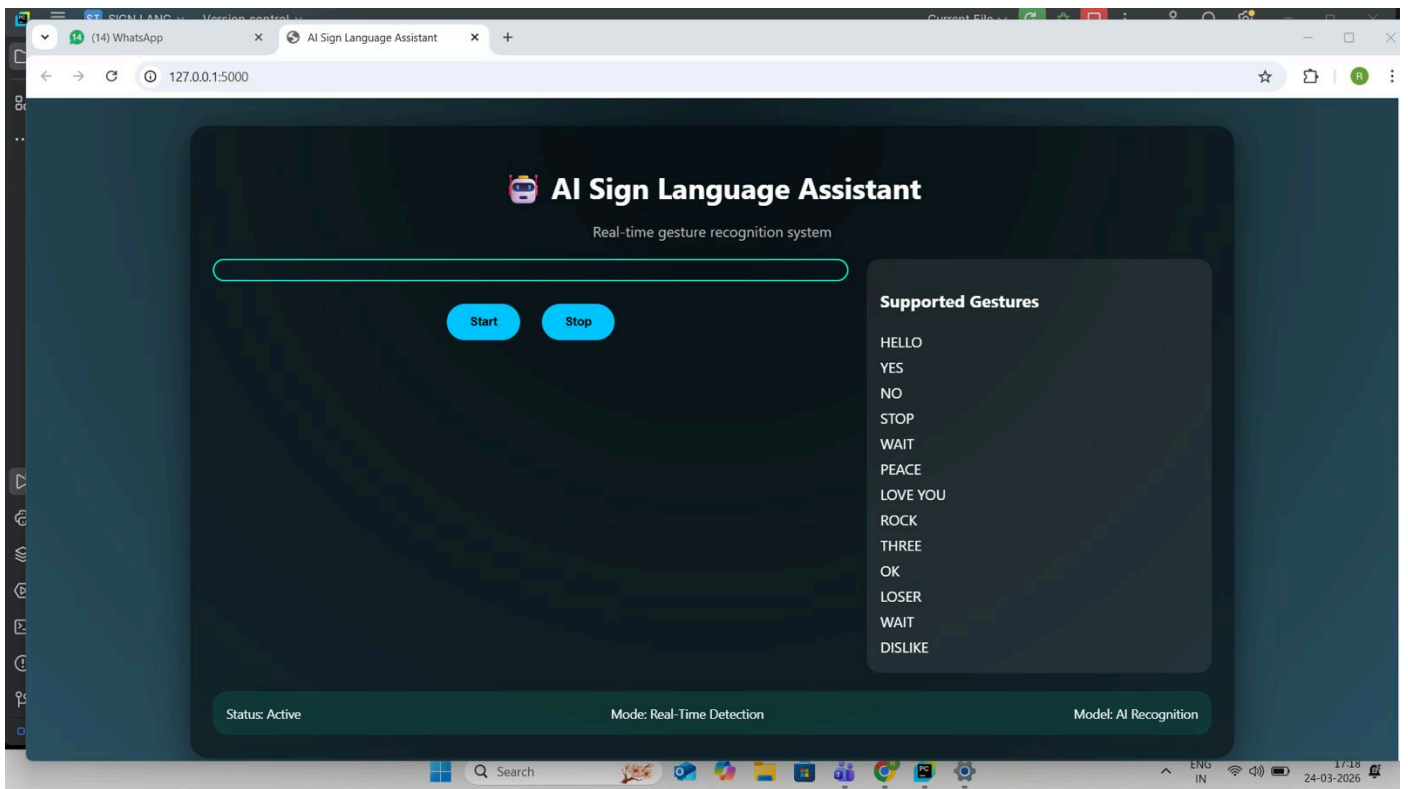
4.2 USER INTERFACE

The User Interface (UI) is the component of the system that allows users to interact with the application in an easy and efficient manner. It is designed with a simple and intuitive layout so that users can operate the system without any difficulty. The interface displays the live video feed captured from the webcam, enabling real-time gesture recognition.

The UI provides basic controls such as Start and Stop buttons to control the execution of the system. It also includes a section that displays the list of supported gestures for user reference. The recognized gesture is dynamically shown on the screen as text output, providing instant feedback to the user.

The interface is developed using web technologies such as HTML and CSS and is integrated with the backend using the Flask framework. This ensures smooth communication between the frontend and the machine learning model, resulting in fast and responsive performance. Overall, the UI is designed to be user-friendly, efficient, and suitable for real-time applications.

4.3 USER INTERFACE DESIGN



CHAPTER 5

METHODOLOGY

5.1 DATA COLLECTION

Data collection is an important step in developing the Sign Language Recognition System. In this project, a dataset of hand gestures is created using a webcam. The system captures multiple samples for each gesture to ensure variation in hand position, angle, and orientation.

A total of 14 different static hand gestures are considered, and each gesture contains approximately 150–200 samples. The data is collected under controlled conditions such as proper lighting, plain background, and consistent hand positioning to reduce noise and improve accuracy. Care is taken to ensure that the gestures are clearly visible and properly aligned within the frame.

The captured gesture data is then stored and used for training the machine learning model. A well-structured dataset helps in achieving better performance and reliable gesture recognition.

5.2 DATA PREPROCESSING

Data preprocessing is an important step to improve the quality of the collected data and ensure better performance of the model. The raw data obtained from the webcam may contain noise and variations that can affect the accuracy of gesture recognition.

The following preprocessing steps are applied:

1. Data Cleaning

The collected landmark data is checked for missing or inconsistent values. Any invalid data is removed to maintain accuracy and reliability.

2. Normalization

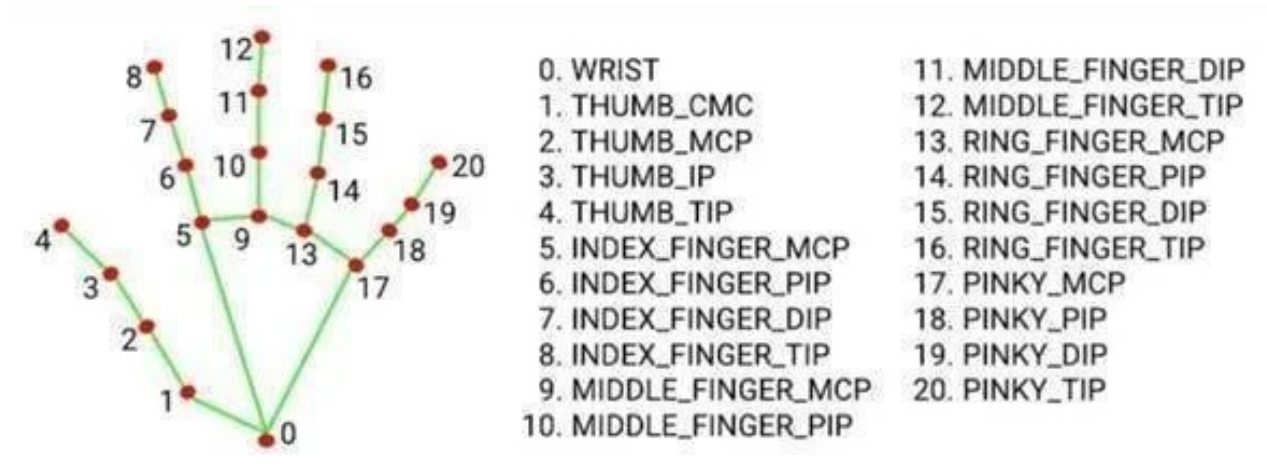
The landmark coordinates are normalized to reduce the effect of variations in hand size, position, and distance from the camera.

3. Noise Reduction

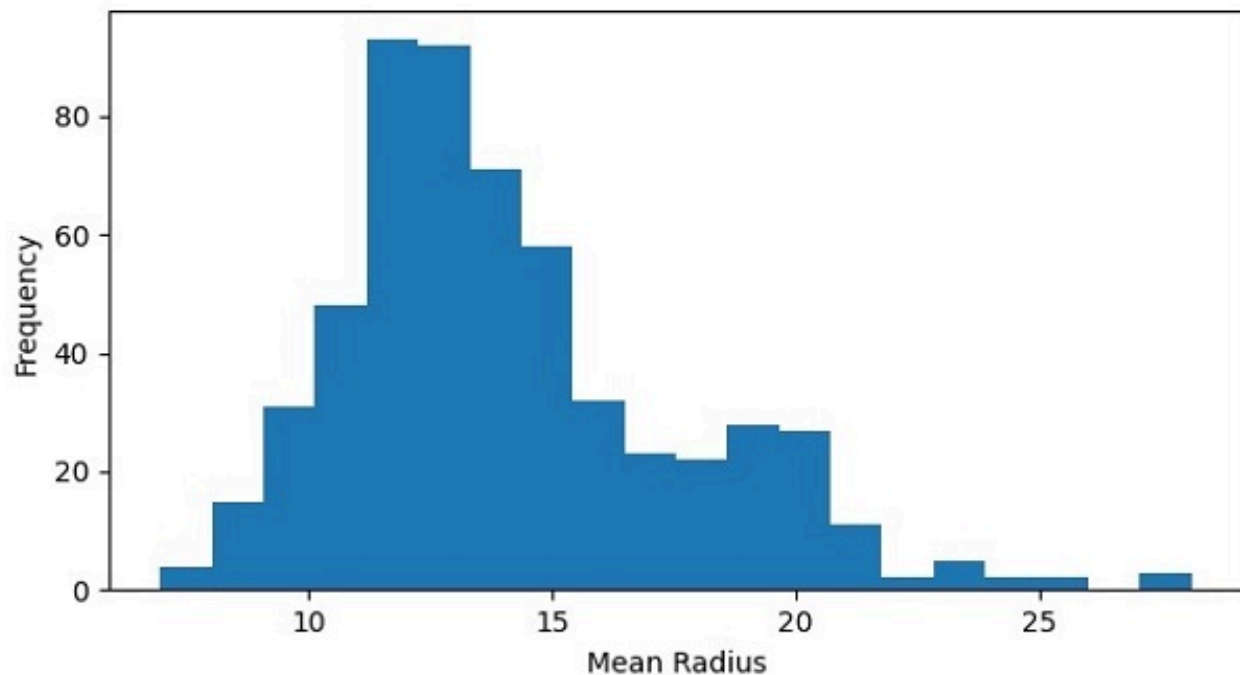
Unwanted variations and fluctuations in the data are minimized to ensure smoother and more stable input for the model.

5.3 DATA VISUALIZATION

Data visualization is an essential step in understanding the structure and behavior of the collected gesture dataset before training the machine learning model. It helps in analyzing patterns, identifying relationships between features, and detecting inconsistencies or outliers in the data. By representing data in graphical form, it becomes easier to interpret complex information and make necessary improvements during preprocessing. In this project, visualization is mainly performed on the hand landmark data extracted using MediaPipe. The system detects 21 key points on the hand, representing the position of fingers and palm. These points provide a structured representation of each gesture, making it easier to analyze how different gestures vary. Visualization techniques such as scatter plots and landmark mapping are used to observe the distribution of coordinate values. These visualizations help in identifying clear differences between gesture patterns and ensure that the data is consistent across different samples. It also helps in detecting outliers or incorrect data points that may affect the model performance.



The above graphs represent the visualization of hand landmarks detected by MediaPipe. Each point corresponds to a specific position on the hand, and the connections form a skeleton-like structure. These visual representations help in clearly understanding the differences between gestures and improve the overall quality of the dataset.



In this project, histograms are used to visualize the distribution of landmark coordinate values extracted from hand gestures. The x-axis represents the range of values, while the y-axis shows the frequency of occurrence. By analyzing the histogram, it is possible to identify whether the data is evenly distributed or skewed. It also helps in detecting outliers and unusual patterns in the dataset. A well-distributed dataset ensures better model training and improves prediction accuracy. Overall, histogram visualization plays an important role in understanding data behavior and improving the quality of preprocessing and model performance.

5.4 MODEL BUILDING

Model building is a crucial phase in the development of the Sign Language Recognition System, where the preprocessed data is used to train a machine learning model for accurate gesture classification. The objective is to develop a model that can learn patterns from the input data and predict the correct gesture in real time.

In this project, different machine learning models are considered for gesture recognition. Initially, a Feedforward Neural Network (FNN) is used due to its simplicity and ability to handle structured input data. The input to the model consists of normalized hand landmark coordinates extracted using MediaPipe.

To improve performance, an advanced model such as Long Short-Term Memory (LSTM) is also implemented. LSTM is suitable for handling sequential data, as it captures temporal dependencies between frames in a gesture sequence. The model takes a sequence of frames as input and processes the landmark coordinates to classify gestures accurately..

The model is trained using labeled data, where each input corresponds to a specific gesture class. During training, the model learns the relationship between input features and output labels. The performance of the model is evaluated using accuracy metrics, and the best-performing model is selected.

Finally, the trained model is integrated into the system to perform real-time gesture prediction and display the output on the user interface.

CHAPTER 6

FUTURE SCOPE

6.1 POTENTIAL IMPROVEMENTS

The Sign Language Recognition System demonstrates the effective use of computer vision and machine learning for real-time gesture recognition. Although the current system is limited to static gestures, there is significant scope for improvement and expansion in the future.

1. Dynamic Gesture Recognition

The system can be extended to recognize continuous hand movements, allowing users to form words and sentences instead of only static gestures.

2. Improved Accuracy

By increasing the dataset size and including more variations in gestures, the overall accuracy and reliability of the model can be enhanced.

3. Voice Output Integration

The system can be integrated with text-to-speech technology to convert recognized gestures into spoken words, improving communication.

4. Support for Multiple Sign Languages

The system can be expanded to support different sign languages such as Indian Sign Language (ISL), making it more versatile.

5. Mobile/Web Application Development

The system can be deployed as a mobile or web application to increase accessibility and usability in real-world scenarios.

CHAPTER 7

IMPLEMENTAION

Implementation is an important phase in the development of the Sign Language Recognition System, where the designed system is converted into a working application. The system is implemented using Python programming language along with libraries such as OpenCV, MediaPipe, and TensorFlow.

The implementation begins with capturing real-time video input using a webcam. OpenCV is used to process the video frames, while MediaPipe is used to detect the hand and extract 21 landmark points. These landmarks are then preprocessed and converted into feature vectors.

A machine learning model such as Feedforward Neural Network (FNN) or Long Short-Term Memory (LSTM) is trained using the collected dataset. The trained model is then integrated into the system for real-time gesture prediction.

The system continuously captures frames, processes the input, and predicts the gesture using the trained model. The predicted output is displayed as text on the user interface. The implementation ensures that the system works efficiently in real time and provides accurate results under suitable conditions.

1. Import Libraries

```
import cv2
import numpy as np
import mediapipe as mp
import os
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from tensorflow.keras.utils import to_categorical
```

2. Initialize MediaPipe

```
mp_hands = mp.solutions.hands
mp_draw = mp.solutions.drawing_utils
```

```
hands = mp_hands.Hands(
    static_image_mode=False,
    max_num_hands=1,
    min_detection_confidence=0.7,
    min_tracking_confidence=0.7
)
```

3. Define Dataset Path and Actions

```
DATA_PATH = "data"
```

```
actions = np.array(['hello', 'yes', 'no', 'stop', 'peace'])
```

```
no_sequences = 150
sequence_length = 30
```

4. Extract Keypoints Function

```
def extract_keypoints(results):
    if results.multi_hand_landmarks:
        hand = results.multi_hand_landmarks[0]
        return np.array([[lm.x, lm.y, lm.z] for lm in hand.landmark]).flatten()
    else:
        return np.zeros(21 * 3)
```

5. Load Data

```
sequences, labels = [], []
```

```
for idx, action in enumerate(actions):
    for sequence in range(no_sequences):
        window = []
        for frame_num in range(sequence_length):
            file_path = os.path.join(DATA_PATH, action, str(sequence),
f'{frame_num}.npy')
            res = np.load(file_path)
            window.append(res)
        sequences.append(window)
        labels.append(idx)
```

```
X = np.array(sequences)
y = to_categorical(labels).astype(int)
```

6. Build LSTM Model

```
model = Sequential()
```

```
model.add(LSTM(64, return_sequences=True, activation='relu',  
input_shape=(30, 63)))
```

```
model.add(LSTM(128, return_sequences=True, activation='relu'))
```

```
model.add(LSTM(64, return_sequences=False, activation='relu'))
```

```
model.add(Dense(64, activation='relu'))
```

```
model.add(Dense(32, activation='relu'))
```

```
model.add(Dense(actions.shape[0], activation='softmax'))
```

7. Compile Model

```
model.compile(  
    optimizer='adam',  
    loss='categorical_crossentropy',  
    metrics=['categorical_accuracy']  
)
```

8. Train Model

```
model.fit(X, y, epochs=50, batch_size=16)
```

9.Real-Time Prediction Code

```
model.load_weights("sign_language_model.h5")

cap = cv2.VideoCapture(0)

sequence = []
sentence = []

THRESHOLD = 0.8

while cap.isOpened():
    ret, frame = cap.read()

    image = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results = hands.process(image)

    if results.multi_hand_landmarks:
        hand = results.multi_hand_landmarks[0]
        mp_draw.draw_landmarks(frame, hand, mp_hands.HAND_CONNECTIONS)

        keypoints = extract_keypoints(results)
        sequence.append(keypoints)
        sequence = sequence[-30:]

    if len(sequence) == 30:
        res = model.predict(np.expand_dims(sequence, axis=0))[0]
        if np.max(res) > THRESHOLD:
            sentence = actions[np.argmax(res)]
```

```
cv2.putText(frame, f'Prediction: {sentence}',  
(10, 40),  
cv2.FONT_HERSHEY_SIMPLEX,  
1,  
(0, 255, 0),  
2)  
cv2.imshow('Sign Language Recognition', frame)  
if cv2.waitKey(10) & 0xFF == ord('q'):  
    break  
cap.release()  
cv2.destroyAllWindows()
```

CHAPTER 8

TEST CASES

TEST CASES:

Testing is a crucial phase in the development of the Sign Language Recognition System, as it ensures that the system performs accurately and reliably under different conditions. It helps in verifying whether the system meets the required objectives and functions as expected in real-time scenarios.

8.1 Purpose of Test Cases

The testing process involves checking the accuracy of gesture recognition, evaluating system performance, and identifying any errors or limitations. It also ensures that the integration between different components such as the webcam, MediaPipe, and the machine learning model works smoothly. The main objectives of testing are to verify the correctness of predictions, ensure real-time responsiveness, and improve the overall efficiency of the system.

8.2 Test Case Report

Test Case	Gesture Class	Input Condition	Expected Output	Result
TC1	HELLO	Good lighting	HELLO	Passed
TC2	STOP	Stable hand position	STOP	Passed
TC3	YES	Proper orientation	YES	Passed
TC4	NO	Clear gesture	NO	Passed
TC5	THANK YOU	Slight variation	THANK YOU	Passed

CHAPTER 9

GESTURES

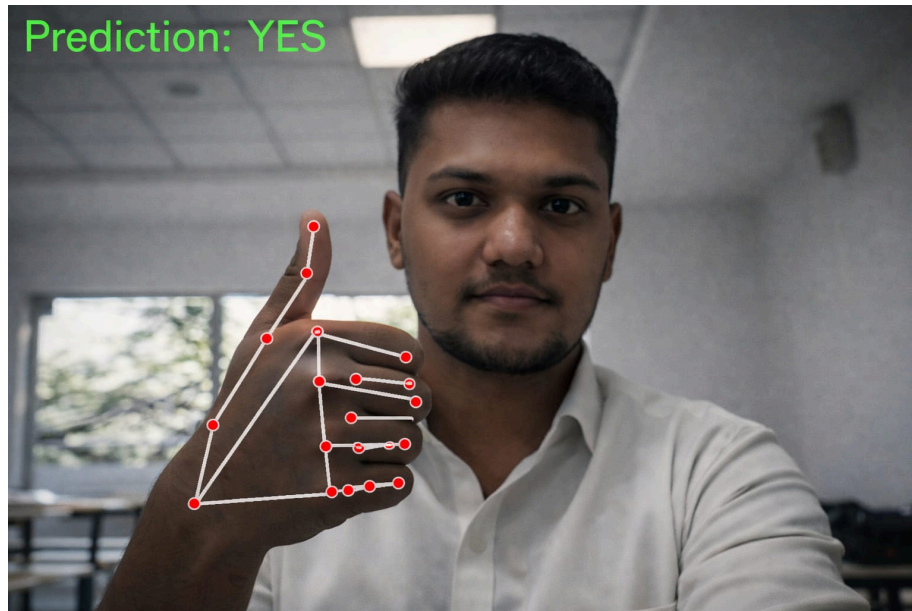


Figure 1.0

The “YES” gesture is performed by moving the hand in an up-and-down motion, similar to nodding the head.

In this system, the gesture is detected using hand tracking and motion analysis techniques.

It is then classified as “YES” and converted into text or speech for communication.

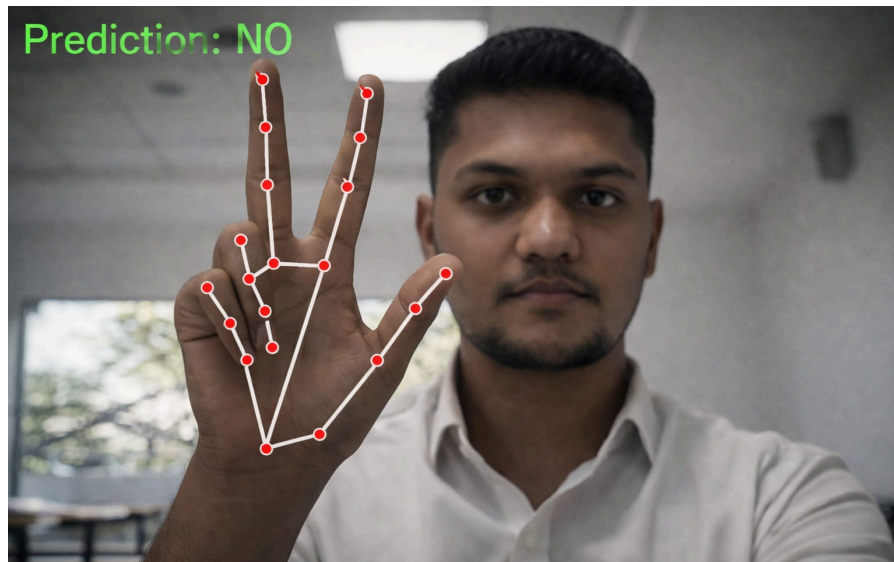


Figure 1.1

The “NO” gesture is performed by moving the hand side-to-side, similar to shaking the head.

The system detects and classifies this motion as “NO” and converts it into text or speech

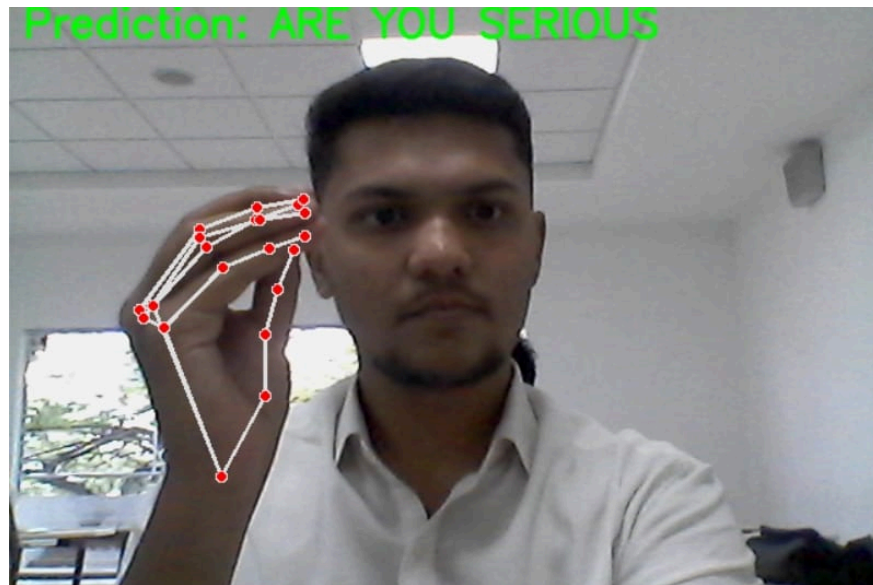


Figure 1.2

The above Figure 1.2 shows The system detects a questioning expression with slight hand movement and classifies it as “ARE YOU SERIOUS,” converting it into text or speech.

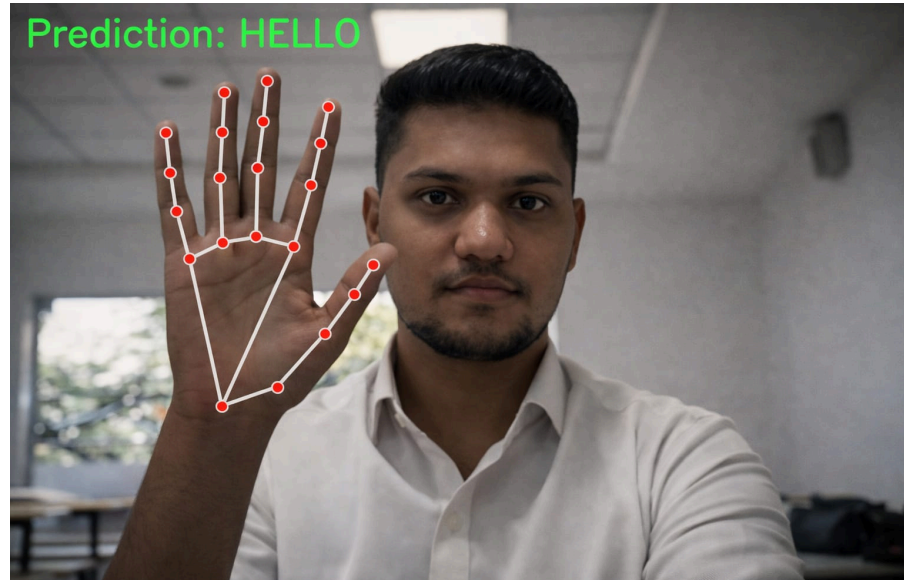


Figure 1.3

The above Figure 1.3 shows the “HELLO” gesture is performed by raising the hand and waving it side-to-side. The system detects this motion and classifies it as “HELLO,” converting it into text or speech

CHAPTER 10

CONCLUSION

The Sign Language Recognition System successfully demonstrates the use of computer vision and machine learning techniques for real-time gesture recognition. The system is capable of accurately identifying predefined hand gestures using MediaPipe and a trained neural network model.

The project follows a structured approach including data collection, preprocessing, model building, and real-time implementation. The system provides fast and efficient performance and is able to convert hand gestures into readable text, helping to reduce the communication gap for hearing and speech impaired individuals.

Although the system performs well under controlled conditions, its accuracy may vary with changes in lighting and gesture variations. With further improvements such as larger datasets, advanced models, and additional features, the system can be enhanced for real-world applications and scalability.

CHAPTER 11

REFERENCES

1. OpenCV. (n.d.). Open Source Computer Vision Library. Retrieved from <https://opencv.org>
2. Google AI. (n.d.). MediaPipe Hands Documentation. Retrieved from <https://developers.google.com/mediapipe>
3. TensorFlow. (n.d.). Machine Learning Framework Documentation. Retrieved from <https://www.tensorflow.org>
4. Géron, A. (2019). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd ed.). O'Reilly Media.
5. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research.
6. W3Schools. (n.d.). HTML, CSS, and JavaScript Tutorials. Retrieved from <https://www.w3schools.com>
7. Kaggle. (n.d.). Sign Language Dataset Resources. Retrieved from <https://www.kaggle.com>
8. Zhang, Z. (2012). Microsoft Kinect Sensor and Its Effect. IEEE Multimedia.
9. Molchanov, P., Gupta, S., Kim, K., & Kautz, J. (2015). Hand Gesture Recognition with 3D CNNs. IEEE Conference on Computer Vision.
10. IEEE Papers on Sign Language Recognition and Gesture Detection Techniques.